МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт (факультет) Компьютерных наук
Кафедра Прикладной математики и системного анализа

ЛАБОРАТОРНАЯ РАБОТА №5

По дисциплине: «Современные платформы разработки ПО».
На тему: «Безопасность данных».

Студент ПМ-24-1 (группа) _____ (подпись, дата) Сараев А.П. (фамилия, инициалы)

Руководитель

к.т.н. (ученая степень, ученое звание) _____ (подпись, дата) Немец С.Ю. (фамилия, инициалы)

Липецк 2026

Цель работы

Необходимо реализовать возможность управления пользователями в рамках предыдущих работ. Вместо открытого хранения паролей требуется реализовать необратимое шифрование паролей с солью. Соль должна создаваться отдельно для каждого пользователя, храниться в файле пользователей.

Задание

Необходимо реализовать функции 1) добавления, 2) изменения пароля и 3) удаления пользователей. Добавление и удаление пользователей делает только admin, изменение пароля у себя может сделать пользователь, admin может выполнить изменение пароля любого пользователя.

Если пользователя admin не существует, при запуске программы он создается, пароль генерируется случайный, информация о создании admin и его пароль при создании выводится в консоль.

Текст программы

Menu.java

```
package org.example;

import org.example.models.LaptopType;
import org.example.services.AuthService;
import org.example.utils.FileManager;
import org.example.utils.ReportGenerator;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import org.example.models.Laptop;
import org.example.services.LaptopService;

import java.io.IOException;
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeParseException;
import java.util.List;
import java.util.Scanner;

public class Menu {
 private static final Logger logger = LoggerFactory.getLogger(Menu.class);

 private Scanner sc;
 private LaptopService laptopService;
 private AuthService authService;
 private FileManager fileManager;
 private String username;
 private String  laptopsFilename;
 private String  usersFilename;
 private ReportGenerator reportGenerator;

 public Menu(Scanner sc, LaptopService laptopService, AuthService authService, FileManager fileManager,
 String username, String laptopsFilename, String usersFilename) {
 this.sc = sc;
 this.laptopService = laptopService;
 this.authService = authService;
 this.fileManager = fileManager;
 this.username = username;
 this.laptopsFilename = laptopsFilename;
 this.reportGenerator = new ReportGenerator();
 this.usersFilename = usersFilename;
 }

 private void showMenu() {
 System.out.println("\nChoose:");
 System.out.println("1 - Show laptops");
 System.out.println("2 - Add laptop");
 System.out.println("3 - Delete laptop");
 System.out.println("4 - Edit laptop");
 System.out.println("5 - Find laptop by model");
 System.out.println("6 - Make report of one laptop");
 System.out.println("7 - Make report of all laptops");
 System.out.println("8 - Change your password");
 if (username.equals("admin")) {
 System.out.println("-----Admin commands-----");
 System.out.println("10 - Add user");
```

```
            System.out.println("11 - Delete user");
            System.out.println("12 - Change other user password");
        }
        System.out.println("0 - Exit");
    }

    int getUserChoice() {
        while (true) {
            try {
                return Integer.parseInt(sc.nextLine());
            } catch (NumberFormatException e) {
                logger.error("Invalid option", e);
                System.out.println("Please enter a valid number");
            }
        }
    }

    public void run() {
        while (true) {
            showMenu();
            int choice = getUserChoice();

            switch (choice) {
                case 1:
                    showLaptops();
                    break;
                case 2:
                    addLaptop();
                    break;
                case 3:
                    deleteLaptop();
                    break;
                case 4:
                    editLaptop();
                    break;
                case 5:
                    findLaptopByModel();
                    break;
                case 6:
                    makeReportOfOneLaptop();
                    break;
                case 7:
                    makeReportOfAllLaptops();
                    break;
                case 8:
                    changeOwnPassword();
                    break;
                case 10:
                    if (username.equals("admin"))
                        addNewUser();
                    else
                        System.out.println("Invalid option");
                    break;
                case 11:
                    if (username.equals("admin"))
                        deleteUser();
                    else
                        System.out.println("Invalid option");
                    break;
                case 12:
                    if (username.equals("admin"))
                        changeOtherUserPassword();
```

```
                else
                    System.out.println("Invalid option");
                break;
            case 0:
                exit();
                return;
            default:
                System.out.println("Invalid option");
        }
    }
}

private static boolean printLaptops(String username, LaptopService laptopService) {
    List<Laptop> userLaptops = laptopService.getUserLaptops(username);
    if (userLaptops.isEmpty()) {
        System.out.println("You have 0 laptops");
        return false;
    }
    for (int i = 0; i < userLaptops.size(); i++) {
        System.out.println((i + 1) + ". " + userLaptops.get(i));
    }
    return true;
}

private void showLaptops() {
    printLaptops(username, laptopService);
}

private void addLaptop() {
    try {
        System.out.print("Model: ");
        String model = sc.nextLine();
        System.out.print("Specifications: ");
        String specs = sc.nextLine();
        System.out.print("Price: ");
        double price = Double.parseDouble(sc.nextLine());
        System.out.print("Enter date (YYYY-MM-DD): ");
        LocalDate date;
        try {
            date = LocalDate.parse(sc.nextLine());
        } catch (DateTimeParseException e) {
            logger.error("Error while parsing date");
            return;
        }

        System.out.print("Enter time (HH:MM): ");
        LocalTime time = LocalTime.parse(sc.nextLine());

        System.out.print("Enter image file name: ");
        String path = sc.nextLine();
        String image = fileManager.encodeImage(path);
        if (image.isEmpty()) {
            return;
        }

        System.out.print("Enter type (OFFICE, GAMING, ULTRABOOK): ");
        LaptopType type = LaptopType.valueOf(sc.nextLine());

        laptopService.addLaptop(username, model, specs,
                price, date, time, image, type);

    } catch (NumberFormatException e) {
```

```
            logger.error("Invalid price", e);
            System.out.println("Price must be number");
        } catch (Exception e) {
            logger.error("Invalid input");
        }
    }

    private void deleteLaptop() {
        if(!printLaptops(username, laptopService)) {
            return;
        }
        System.out.print("Enter laptop number to delete: ");
        try {
            int index = Integer.parseInt(sc.nextLine()) - 1;
            if (!laptopService.deleteLaptop(username, index)) {
                System.out.println("Laptop with this number was not found");
            } else {
                System.out.println("Laptop deleted");
            }
        } catch (NumberFormatException e) {
            logger.error("Invalid input", e);
            System.out.println("Invalid input");
        }
    }

    private void editLaptop() {
        if(!printLaptops(username, laptopService)) {
            return;
        }
        System.out.print("Enter laptop number to edit: ");
        try {
            int index = Integer.parseInt(sc.nextLine()) - 1;
            System.out.print("Model: ");
            String model = sc.nextLine();
            System.out.print("Specifications: ");
            String specs = sc.nextLine();
            System.out.print("Price: ");
            double price = Double.parseDouble(sc.nextLine());
            System.out.print("Enter date (YYYY-MM-DD): ");
            LocalDate date = LocalDate.parse(sc.nextLine());

            System.out.print("Enter time (HH:MM): ");
            LocalTime time = LocalTime.parse(sc.nextLine());

            System.out.print("Enter image file name: ");
            String path = sc.nextLine();
            String image = fileManager.encodeImage(path);

            System.out.print("Enter type (OFFICE, GAMING, ULTRABOOK): ");
            LaptopType type = LaptopType.valueOf(sc.nextLine());

            if (!laptopService.editLaptop(username, index, model, specs, price, date, time, image, type)) {
                System.out.println("Laptop with this number was not found");
            } else {
                System.out.println("Laptop updated");
            }
        } catch (NumberFormatException e) {
            logger.error("Invalid input", e);
            System.out.println("Invalid input");
        } catch (Exception e) {
            logger.error("Invalid input", e);
        }
```

```
}

private void findLaptopByModel() {
    System.out.print("Model: ");
    String model = sc.nextLine();
    try {
        Laptop laptop = laptopService.getLaptopByModel(username, model);
        System.out.println(laptop);
    } catch (Exception e) {
        logger.error("Error while searching laptop", e);
    }
}

private void makeReportOfOneLaptop() {
    if(!printLaptops(username, laptopService)) {
        return;
    }
    System.out.print("Enter laptop number for report: ");
    try {
        int index = Integer.parseInt(sc.nextLine()) - 1;
        reportGenerator.generateOneLaptopReport(laptopService.getLaptopByIndex(username, index));
    } catch (IOException e) {
        logger.error("Error while making report for one laptop", e);
    }
}

private void makeReportOfAllLaptops() {
    try {
        reportGenerator.generateAllLaptopsReport(username, laptopService.getUserLaptops(username));
    } catch (IOException e) {
        logger.error("Error while making report for all laptops", e);
    }
}

private void changeOwnPassword() {
    String newPassword;
    System.out.print("Type your new password: ");
    newPassword = sc.nextLine();
    authService.changePassword(username, newPassword);
    System.out.println("Successfully changed password!");
}

private void addNewUser() {
    System.out.print("Username: ");
    String newUsername = sc.nextLine();
    System.out.print("Password: ");
    String newPassword = sc.nextLine();

    if (!authService.addUser(newUsername, newPassword)) {
        System.out.println("User with such name already exists");
        logger.error("Username {} already exists", username);
    }
    else {
        System.out.println("Successfully added new user");
    }
}

private void deleteUser() {
    System.out.print("Username: ");
    String otherUsername = sc.nextLine();
    if (otherUsername.equals("admin")) {
        System.out.println("admin can't be deleted");
```

```
            logger.error("admin can't be deleted");
        }
        if (authService.deleteUser(otherUsername)) {
            System.out.println("User " + otherUsername + " successfully deleted");
        }
        else {
            System.out.println("User with such name doesn't exists");
            logger.error("User {} doesn't exists", username);
        }
    }

    private void changeOtherUserPassword() {
        System.out.print("Username: ");
        String otherUsername = sc.nextLine();
        System.out.print("Type your new password: ");
        String otherPassword = sc.nextLine();
        if (authService.changePassword(otherUsername, otherPassword))
            System.out.println("Successfully changed password!");
        else {
            System.out.println("User doesn't exists");
            logger.error("User {} doesn't exists", username);
        }
    }

    private void exit() {
        fileManager.saveLaptops(laptopService.getLaptops(), laptopsFilename);
        fileManager.saveUsers(authService.getUsers(), usersFilename);
        System.out.println("Data saved");
    }
}
```

AuthService.java

```
package org.example.services;

import at.favre.lib.crypto.bcrypt.BCrypt;
import lombok.Getter;
import org.apache.commons.lang3.RandomStringUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.example.models.User;


import java.util.Map;

public class AuthService {
 @Getter
 private Map<String, User> users;

 private static final Logger logger =
 LoggerFactory.getLogger(AuthService.class);

 public AuthService(Map<String, User> users) {
 this.users = users;
 if (users.get("admin") == null) {
 String characters = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$%^&";
 String adminPassword = RandomStringUtils.random( 5, characters );
 String adminPasswordHashed = BCrypt.withDefaults().hashToString(12, adminPassword.toCharArray());
 users.put("admin",
 new User("admin", adminPasswordHashed));
            System.out.println("Admin was born!\nUsername: admin\nPassword: " + adminPassword);
```

```
        }
    }

    public boolean login(String username, String password) {
        User user = users.get(username);
        if (user == null)
            return false;
        return BCrypt.verifyer().verify(password.toCharArray(), user.getPassword()).verified;
    }

    public boolean addUser(String username, String password) {
        if (users.get(username) != null) return false;
        users.put(username, new User(username,
                BCrypt.withDefaults().hashToString(12, password.toCharArray())));
        return true;
    }

    public boolean changePassword(String username, String newPassword) {
        User user = users.get(username);
        if (user == null)
            return false;
        user.setPassword(BCrypt.withDefaults().hashToString(12, newPassword.toCharArray()));
        return true;
    }

    public boolean deleteUser(String username) {
        if (users.get(username) == null || username.equals("admin")) return false;
        users.remove(username);
        return true;
    }
}
```

FileManager.java

```
package org.example.utils;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import org.example.models.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.io.*;
import java.util.*;

import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Base64;

public class FileManager {
    private static final Logger logger =
            LoggerFactory.getLogger(FileManager.class);

    public String encodeImage(String path) {
        try {
            byte[] fileContent = Files.readAllBytes(Paths.get(path));
            return Base64.getEncoder().encodeToString(fileContent);
        } catch (Exception e) {
            logger.error("Error encoding image");
            return "";
        }
```

```
}

public Map<String, User> loadUsers(String filename) {
    Map<String, User> users = new HashMap<>();
    try {
        ObjectMapper mapper = new ObjectMapper();
        List<User> userList = mapper.readValue(
                new File(filename),
                new TypeReference<List<User>>() {}
        );
        for (User u : userList) {
            users.put(u.getUsername(), u);
        }
    } catch (Exception e) {
        logger.error("Error loading users", e);
    }
    return users;
}

public List<Laptop> loadLaptops(String filename) {
    File file = new File(filename);

    if (!file.exists() || file.length() == 0) {
        return new ArrayList<>();
    }

    try {
        ObjectMapper mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule());
        return mapper.readValue(
                file,
                new TypeReference<List<Laptop>>() {}
        );
    } catch (Exception e) {
        logger.error("Error loading laptops", e);
        return new ArrayList<>();
    }
}

public void saveLaptops(List<Laptop> laptops, String filename) {
    try {
        ObjectMapper mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule());
        mapper.writerWithDefaultPrettyPrinter()
                .writeValue(new File(filename), laptops);
    } catch (Exception e) {
        logger.error("Error saving laptops", e);
    }
}

public void saveUsers(Map<String, User> usersMap, String filename) {
    try {
        List<User> users = new ArrayList<>();
        usersMap.forEach((key, value) -> {
            users.add(value);
        });
        ObjectMapper mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule());
        mapper.writerWithDefaultPrettyPrinter()
                .writeValue(new File(filename), users);
    } catch (Exception e) {
        logger.error("Error saving users", e);
```

```
    }
  }
}
```

Результаты программы на тестовых данных

```
> Task :org.example.Main.main()
Admin was born!
Username: admin
Password: %9n#C
```

Рисунок 1 — создание admin

```
Choose:
1 - Show laptops
2 - Add laptop
3 - Delete laptop
4 - Edit laptop
5 - Find laptop by model
6 - Make report of one laptop
7 - Make report of all laptops
8 - Change your password
-----Admin commands-----
10 - Add user
11 - Delete user
12 - Change other user password
0 - Exit
8
Type your new password: 123
Successfully changed password!
```

Рисунок 2 — изменение пароля

```
Choose:
1 - Show laptops
2 - Add laptop
3 - Delete laptop
4 - Edit laptop
5 - Find laptop by model
6 - Make report of one laptop
7 - Make report of all laptops
8 - Change your password
-----Admin commands-----
10 - Add user
11 - Delete user
12 - Change other user password
0 - Exit
10
Username: JonyJon
Password: qwerty123
Successfully added new user
```

Рисунок 3 — добавление пользователя


```
[ {
  "username" : "JonyJon",
  "password" : "$2a$12$G1oNqRLK0jLmlhdr0mYw0eGHb4mwQmvLvhykqVlVIFZ1OsPg/Uegy"
}, {
  "username" : "admin",
  "password" : "$2a$12$Q6c9v1.W2pE3SeRT81FMKe.XTTysQGjpoffpZmjYe.wpblcBCtqDy"
}, {
  "username" : "testUser",
  "password" : "$2a$12$USnBEIKb2us54ibIClPLlOBCJDkJcwPj1wD0I9XyM4INyqPvWIpIO"
} ]
```


Рисунок 4 — файл users.json

Вывод

В результате работы была реализована возможность управления пользователями, изменение паролей и шифрование паролей с солью.